PreliminarySims

Walter Navarro

2026-11-13

```
# library
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.5.3
```

This code is for the simulated data based off some factor models as seen in Duker and Pipiras (2024). The main idea of the first chunk of code is to set up our data based off of the factor model set-up we have been given.

```
# set seed
set.seed(293) # Used class number as the seed

# Define the factor models

# Number of variables, observations, and factors
d <- 200
T <- 200
r <- 3

# Significance level
alpha <- 0.05

# Magnitudes of departure from the null
delta_values <- c(0, 0.25, 0.50, 1)

# Use delta = 0.25 for the example
example_j <- which(delta_values == 0.25)

# First loading matrix and a random perturbation matrix
Lambda1 <- matrix(rnorm(d * r), nrow = d, ncol = r)
B <- matrix(rnorm(d * r), nrow = d, ncol = r)

# Nonsingular transformation: rescales the three loading columns
Phi <- diag(c(0.8, 1, 1.2))

# Generate independent standard normal factors
F1 <- matrix(rnorm(T * r), nrow = T, ncol = r)
F2 <- matrix(rnorm(T * r), nrow = T, ncol = r)

# Generate independent standard normal errors
E1 <- matrix(rnorm(T * d), nrow = T, ncol = d)
```

```r
E2 <- matrix(rnorm(T * d), nrow = T, ncol = d)

# Generate the first dataset and subtract each column's mean
X1 <- F1 %*% t(Lambda1) + E1
X1 <- scale(X1, center = TRUE, scale = FALSE)

# Store one loading matrix and dataset for each delta
Lambda2 <- vector("list", length(delta_values))
X2 <- vector("list", length(delta_values))

for (j in seq_along(delta_values)) {
  delta <- delta_values[j]

  # delta = 0 satisfies the null; positive delta adds a perturbation
  Lambda2[[j]] <- Lambda1 %*% Phi + delta * B

  # Generate and center the second dataset
  X2[[j]] <- F2 %*% t(Lambda2[[j]]) + E2
  X2[[j]] <- scale(X2[[j]], center = TRUE, scale = FALSE)
}
```

Out of curiosity. I wanted to see the vizualization of the data that was simulated

```r
# Visualization of Data

# Calculate the projection matrix for the first loading space.
Q1 <- qr.Q(qr(Lambda1))
P1 <- Q1 %*% t(Q1)

# Calculate P2 - P1 for each delta.
projection_differences <- vector("list", length(delta_values))

for (j in seq_along(delta_values)) {
  Q2 <- qr.Q(qr(Lambda2[[j]]))
  P2 <- Q2 %*% t(Q2)

  projection_differences[[j]] <- P2 - P1
}

# The first 3 factors
series_colors <- c("steelblue", "firebrick", "darkgreen")

# Use the same vertical limits across all series plots.
series_limits <- range(
  unlist(lapply(X2, function(X) X[, 1:3]))
)

# Display the first 30 variables in each projection heatmap.
n_show <- min(30, d)

# Use the same color scale across all projection plots.
# Blue = negative difference; white = zero; red = positive difference.
projection_colors <- colorRampPalette(
```

```r
  c("blue", "white", "red")
)(101)

color_limit <- max(abs(unlist(projection_differences)))
color_limit <- max(color_limit, .Machine$double.eps)


#Function for one observed-series plot

# j identifies the delta scenario to display.
plot_series <- function(j) {
  matplot(
    1:T,
    X2[[j]][, 1:3],
    type = "l",
    lty = 1,
    col = series_colors,
    ylim = series_limits,
    xlab = "Time",
    ylab = "Observed value",
    main = bquote(delta == .(delta_values[j]))
  )

  legend(
    "topright",
    legend = paste("Series", 1:3),
    col = series_colors,
    lty = 1,
    bty = "n",
    cex = 0.7
  )
}


# Function for one projection-difference plot

plot_projection <- function(j) {
  image(
    1:n_show,
    1:n_show,
    projection_differences[[j]][1:n_show, 1:n_show],
    col = projection_colors,
    zlim = c(-color_limit, color_limit),
    xlab = "Variable",
    ylab = "Variable",
    main = bquote(P[2] - P[1] ~ ":" ~ delta == .(delta_values[j]))
  )
}


# Display all layouts

show_figures <- function(plot_function) {
```

```r
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))

  # Find the scenario corresponding to delta = 0.
  zero <- which(delta_values == 0)

  # Each delta as a separate figure.
  par(mfrow = c(1, 1), mar = c(4, 4, 3, 1))

  for (j in seq_along(delta_values)) {
    plot_function(j)
  }

  # All four deltas in a 2 x 2 figure.
  par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))

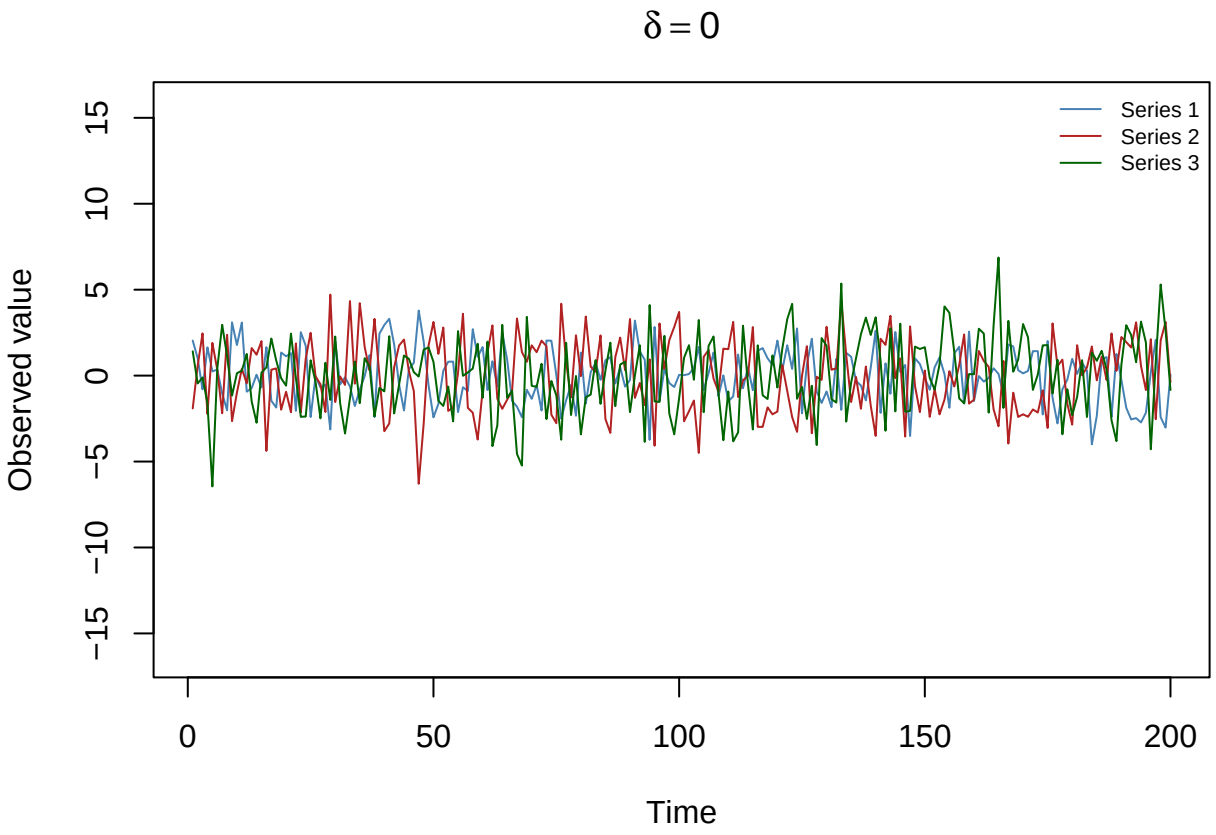
  for (j in seq_along(delta_values)) {
    plot_function(j)
  }

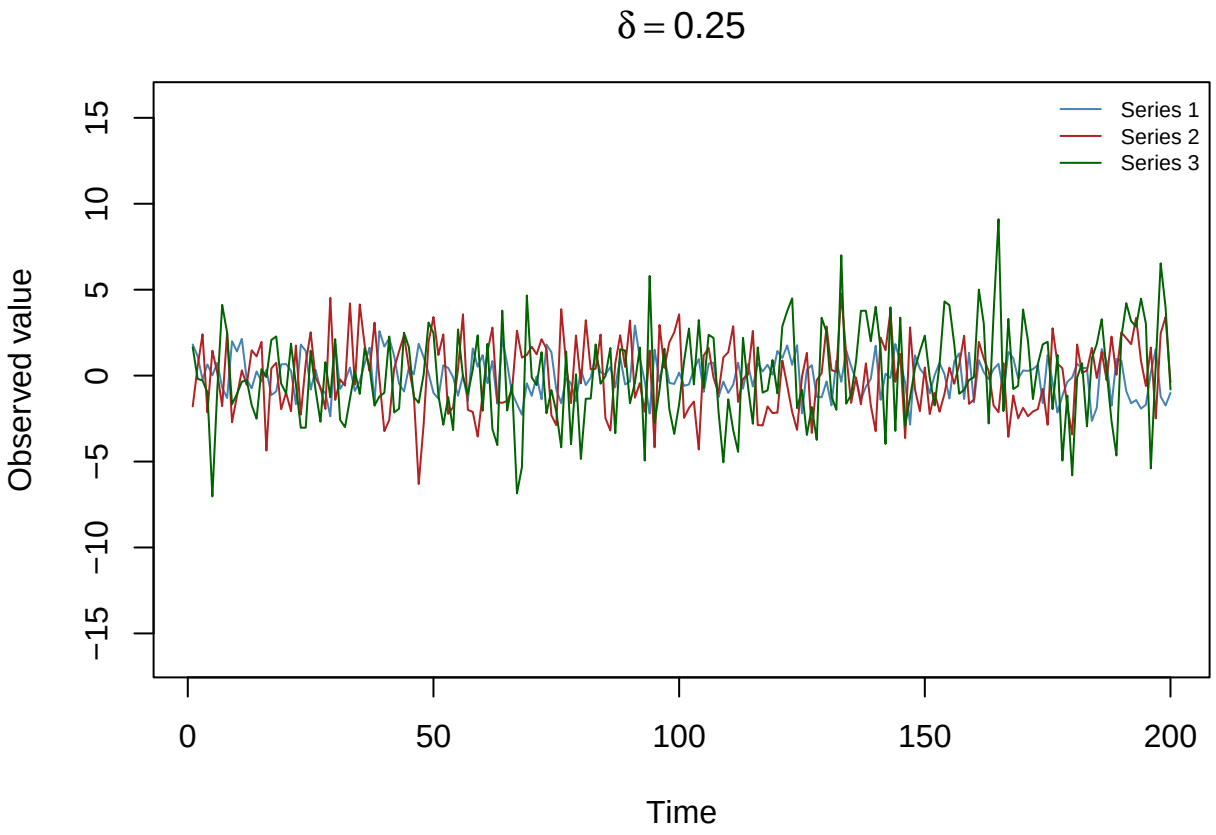
  # Compare delta = 0 with each positive delta (Pairs)
  for (j in which(delta_values > 0)) {
    par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

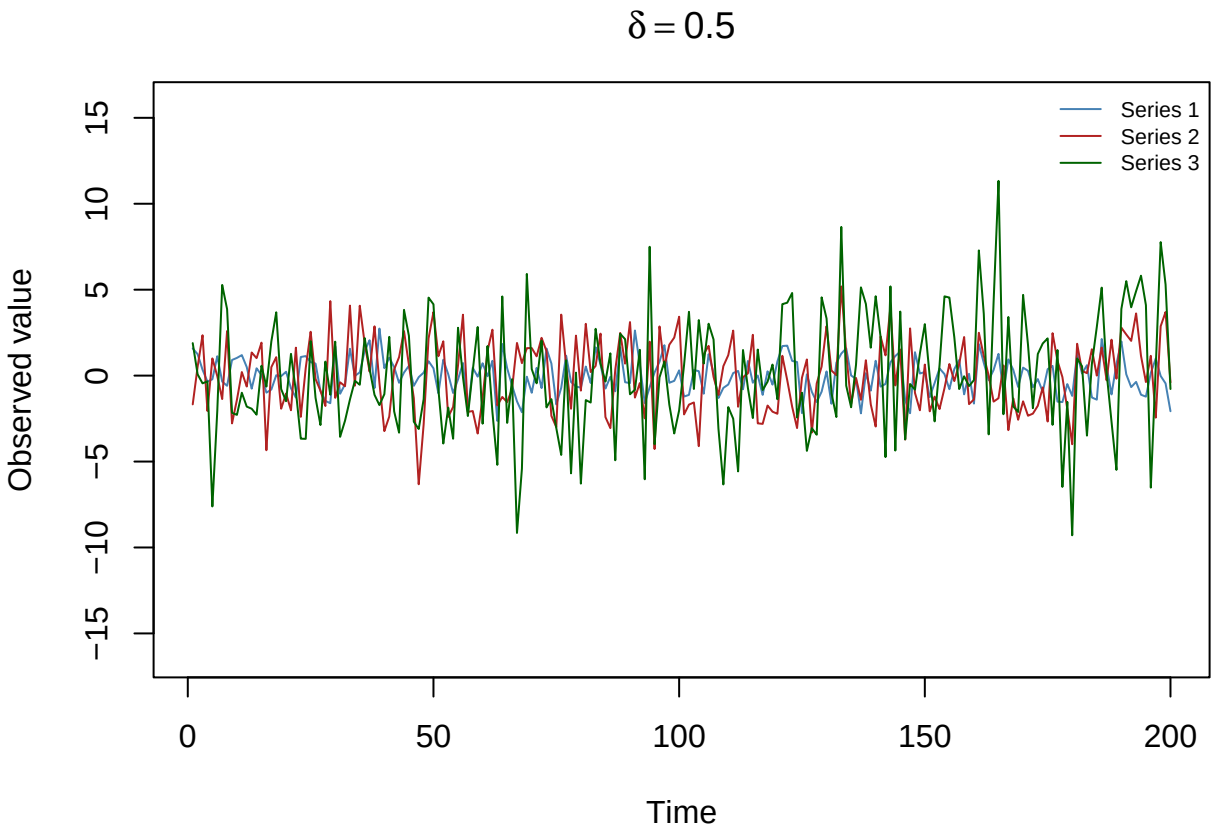
    plot_function(zero)
    plot_function(j)
  }
}

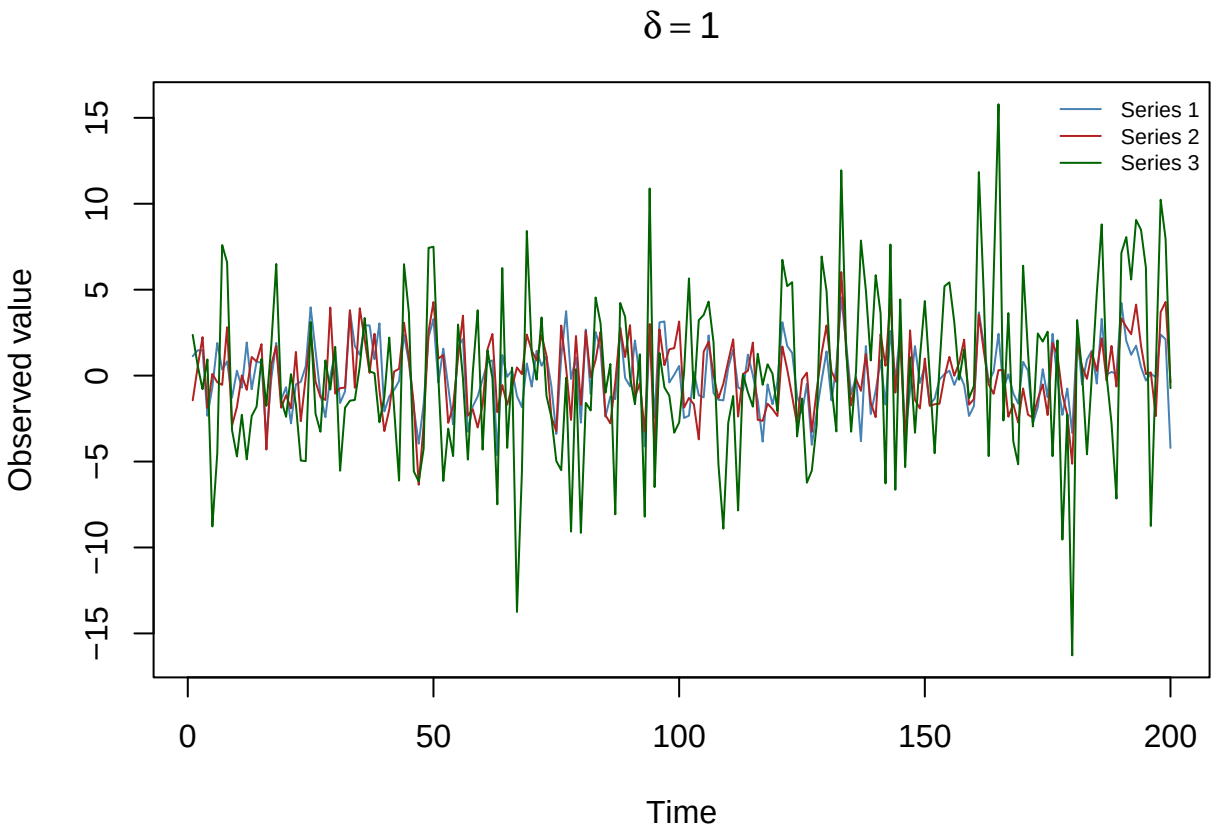

#Display data figures

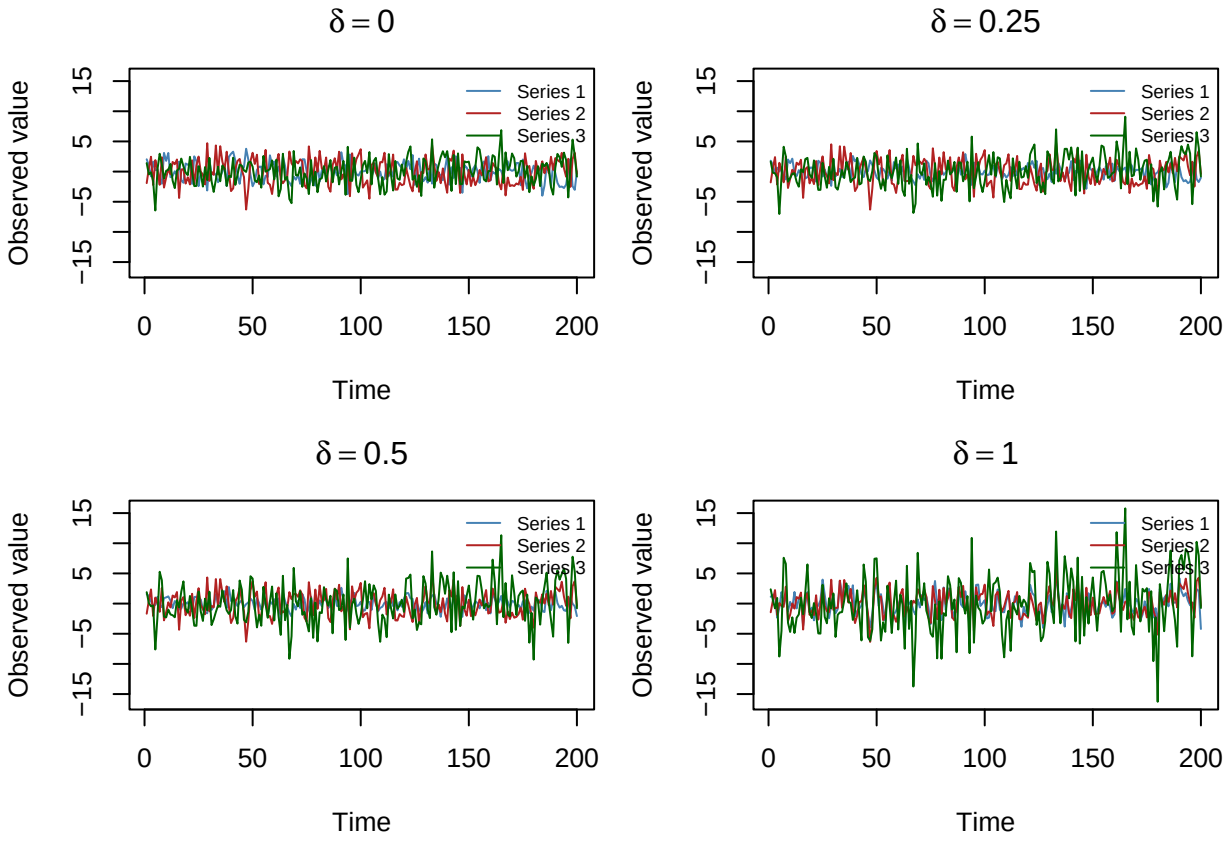
show_figures(plot_series)
```

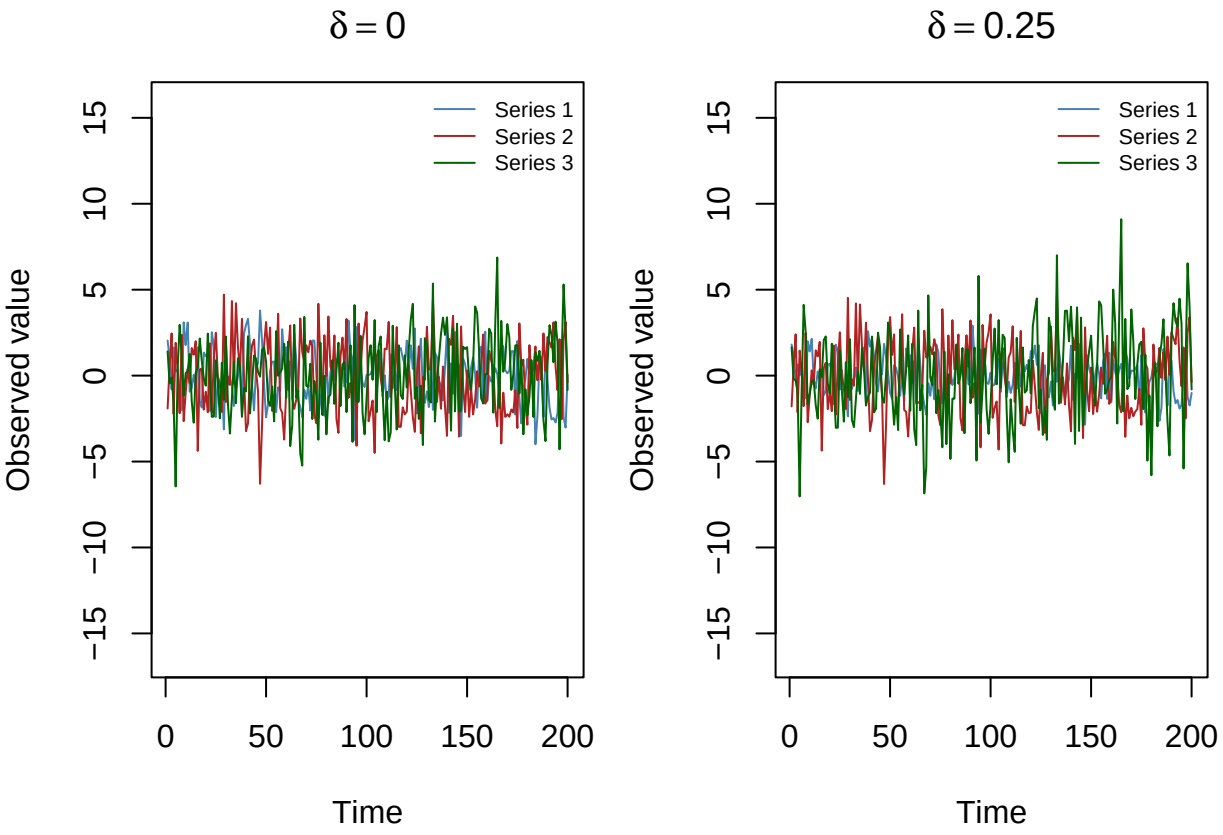

δ = 0
Series 1
Series 2
Series 3
Observed value
15
10
5
0
−5
−10
−15
0
50
100
150
200
Time

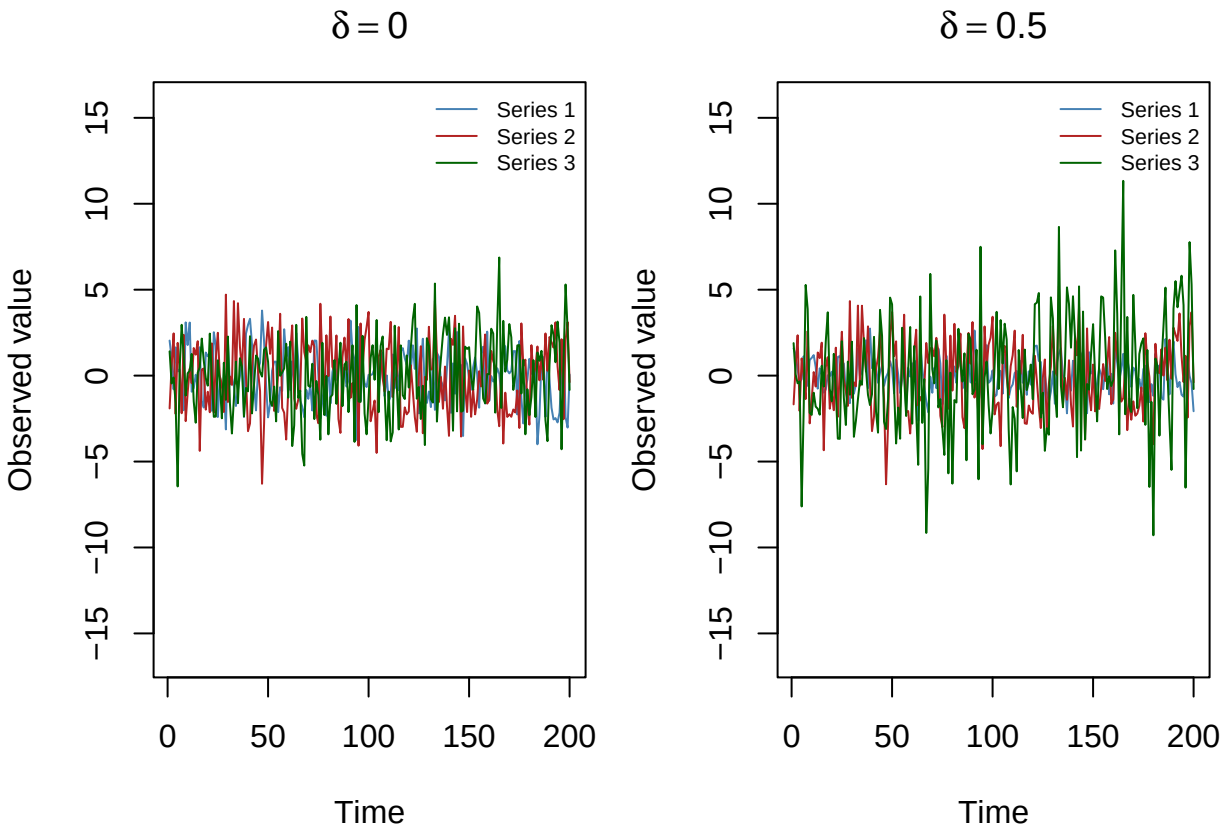

δ = 0.25
Series 1
Series 2
Series 3
Observed value
15
10
5
0
−5
−10
−15
0
50
100
150
200
Time

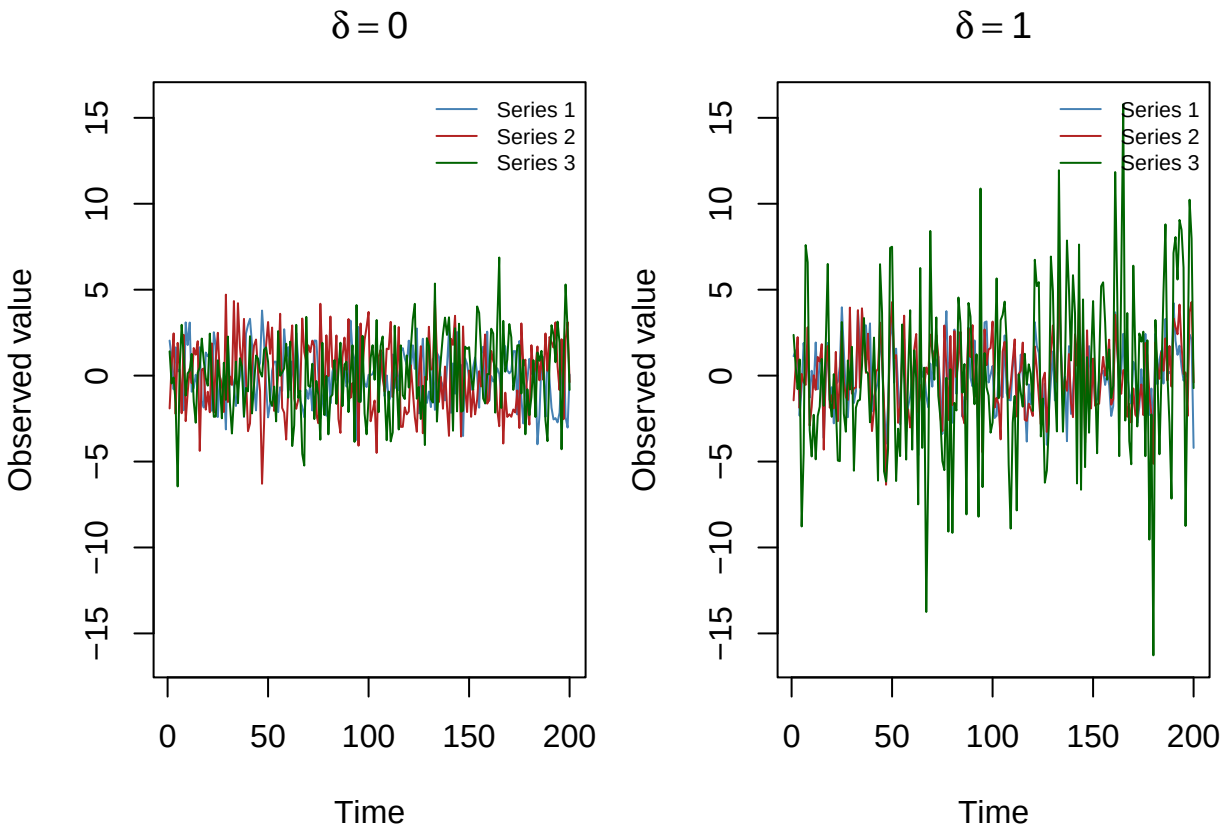

δ = 0.5
Series 1
Series 2
Series 3
Observed value
15
10
5
0
−5
−10
−15
0
50
100
150
200
Time


δ = 1
Series 1
Series 2
Series 3
Observed value
15
10
5
0
−5
−10
−15
0
50
100
150
200
Time


δ = 0
δ = 0.25
δ = 0.5
δ = 1
Observed value
Time
Series 1
Series 2
Series 3
−15
−5
5
15
0
50
100
150
200

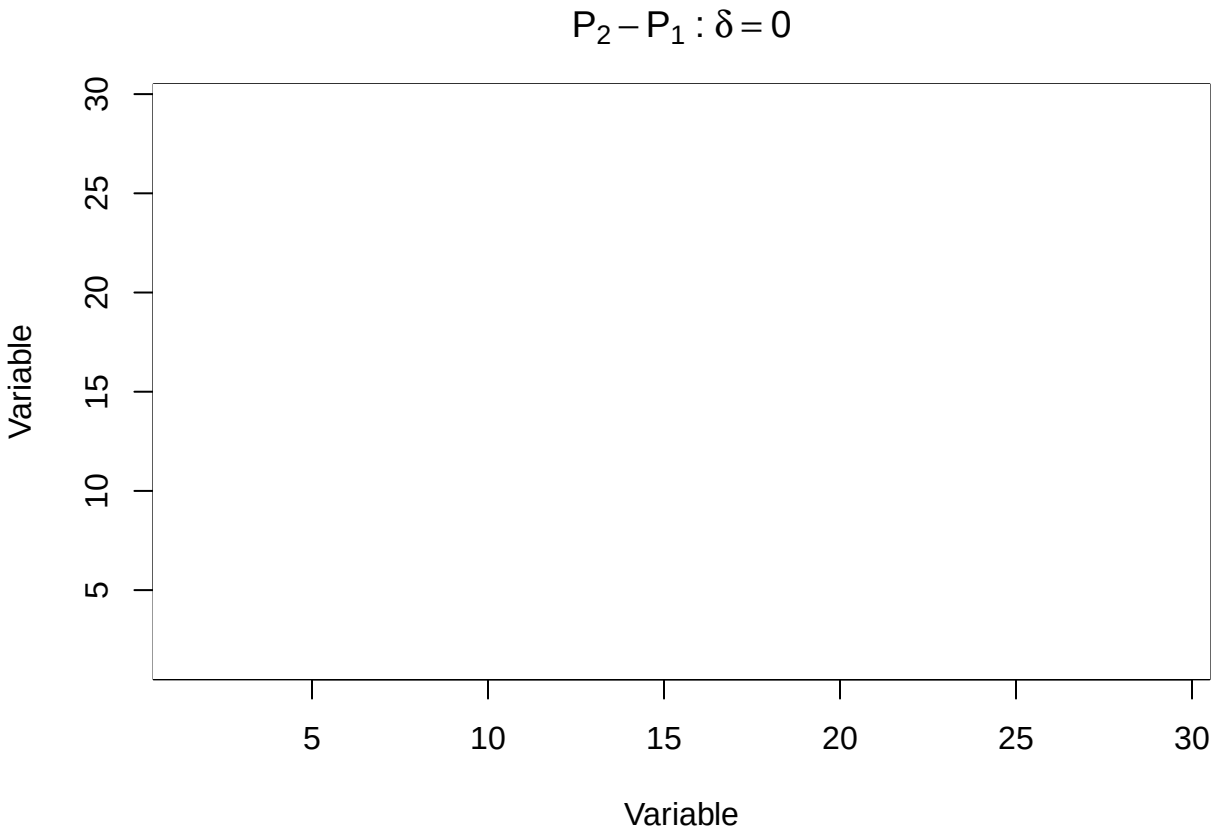

δ = 0
δ = 0.25
Series 1
Series 2
Series 3
Observed value
Time
−15
−10
−5
0
5
10
15
0
50
100
150
200

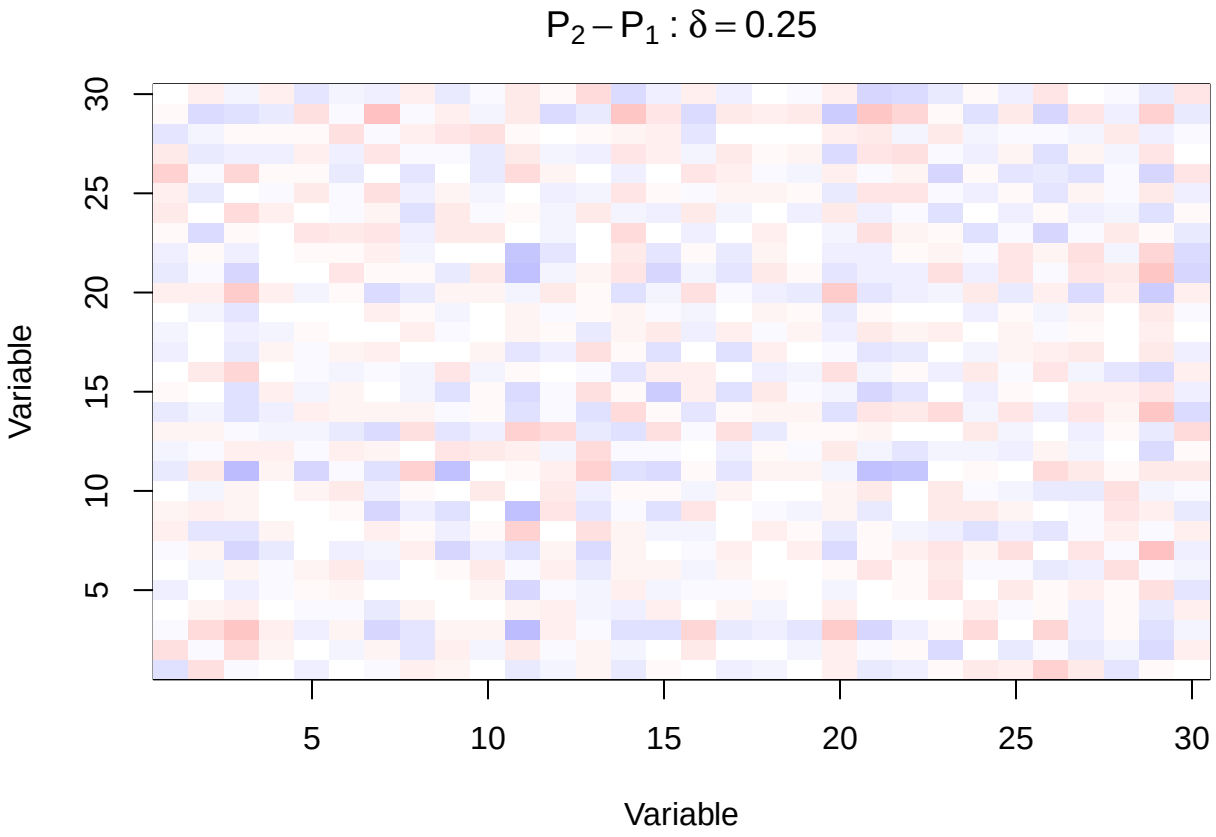

δ = 0
δ = 0.5
Series 1
Series 2
Series 3
Observed value
Time
−15
−10
−5
0
5
10
15
0
50
100
150
200

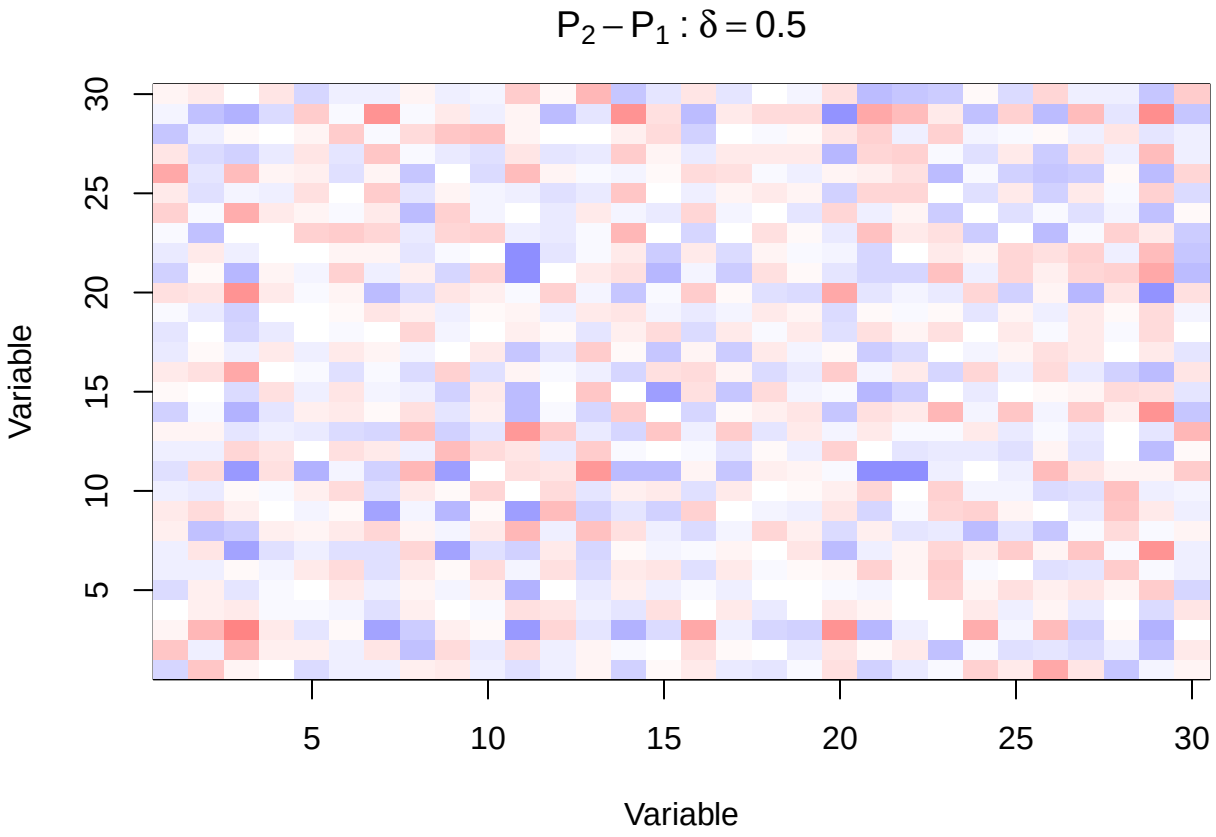


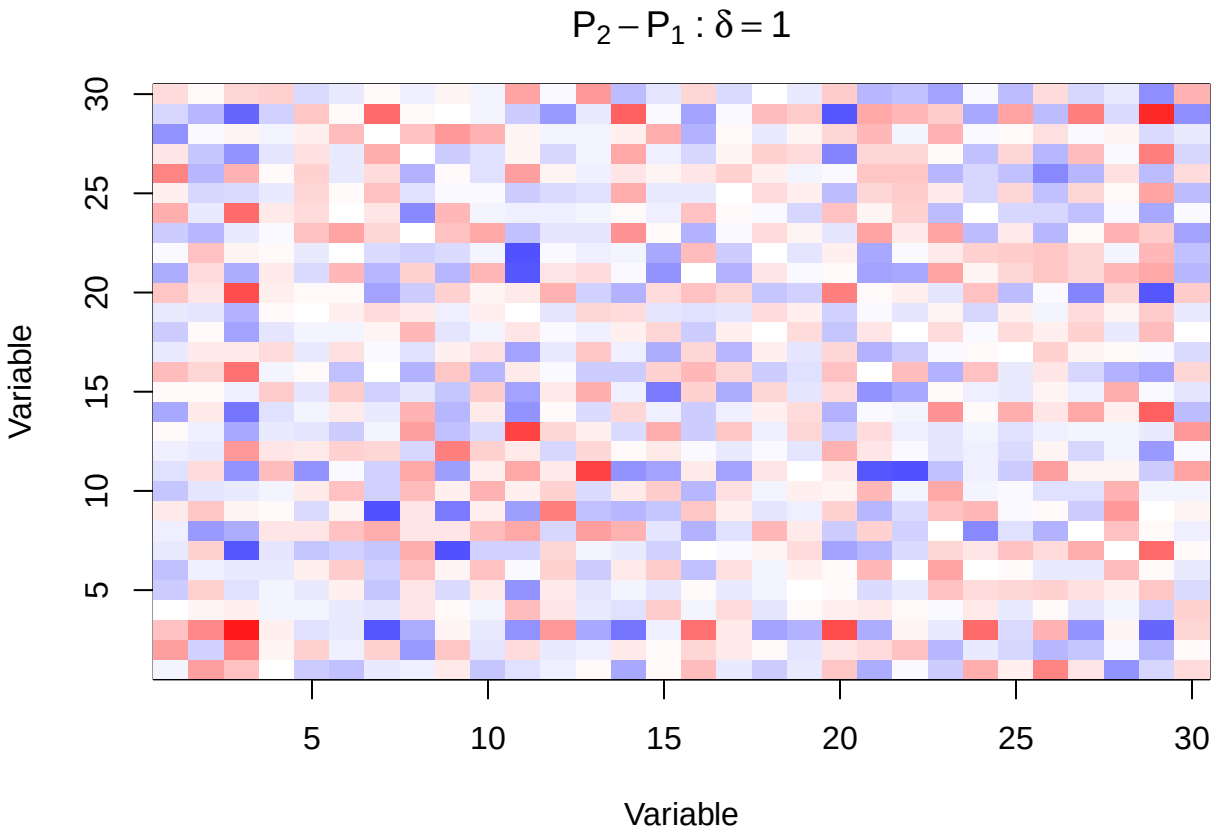

```
# Display the heatmap figures

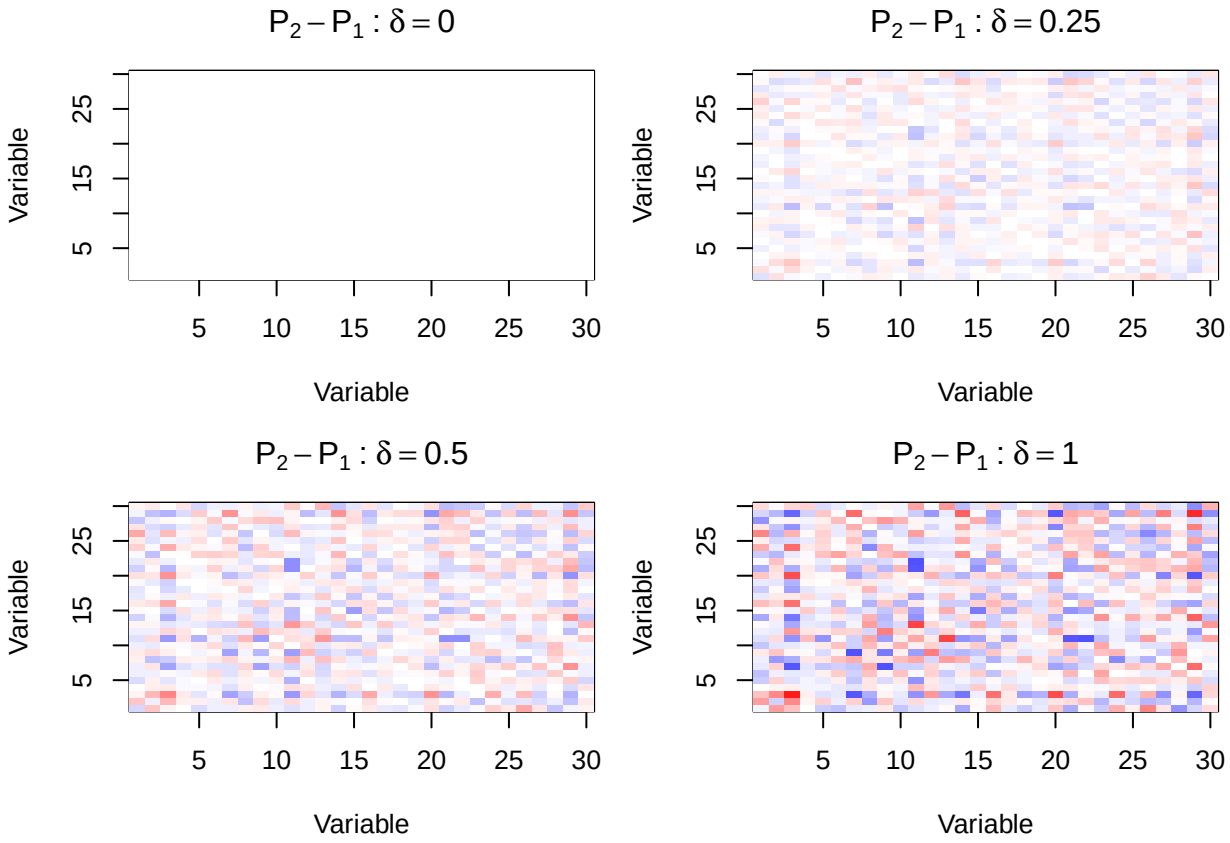
show_figures(plot_projection)
```

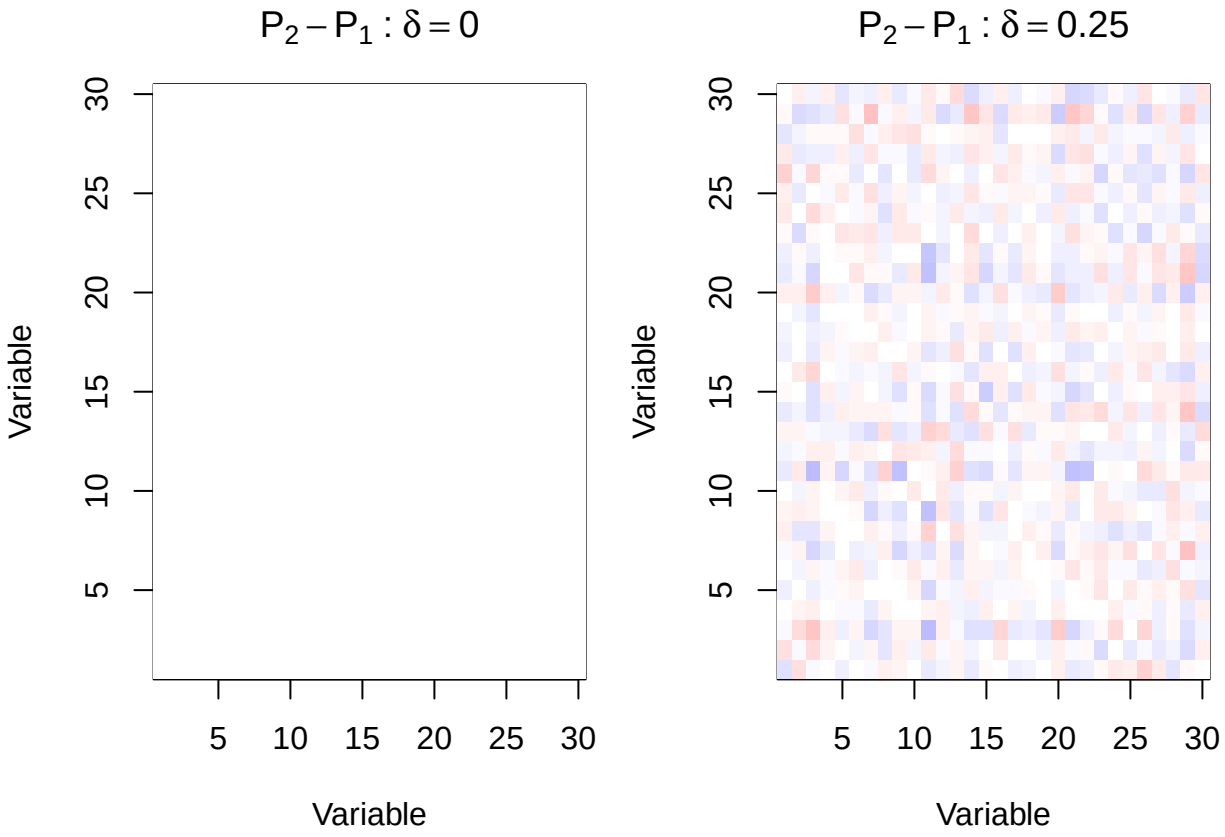

$P_2 - P_1 : \delta = 0$
Variable
5
10
15
20
25
30
Variable
5
10
15
20
25
30

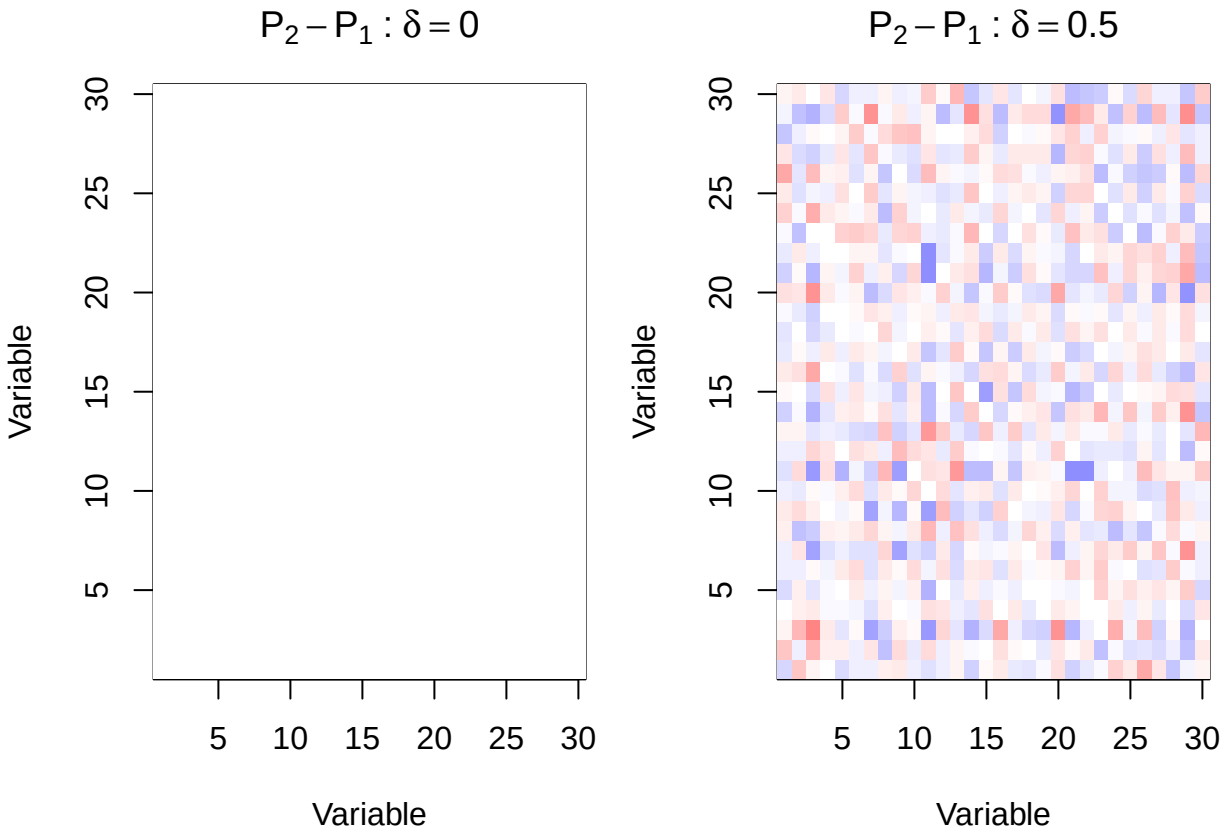

$P_2 - P_1 : \delta = 0.25$
Variable
Variable
5
10
15
20
25
30

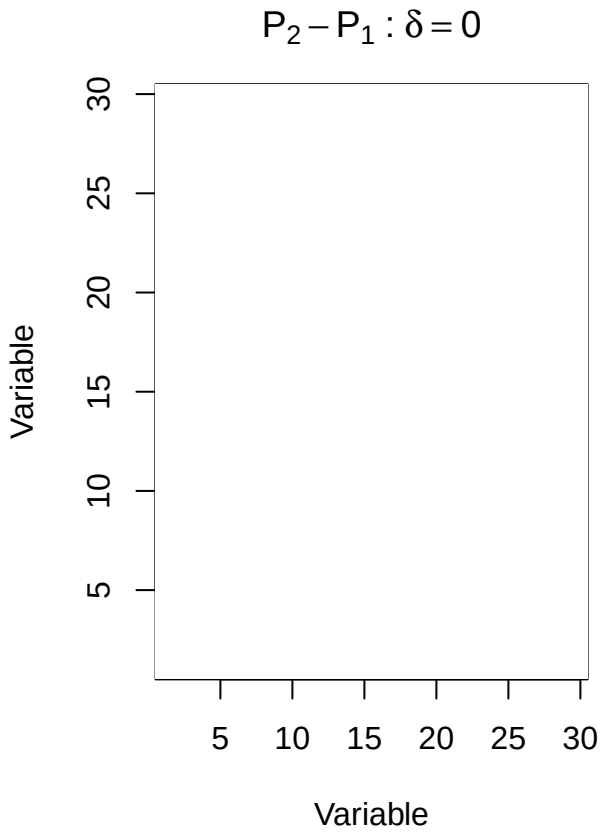

$P_2 - P_1 : \delta = 0.5$
Variable
5
10
15
20
25
30
Variable
5
10
15
20
25
30

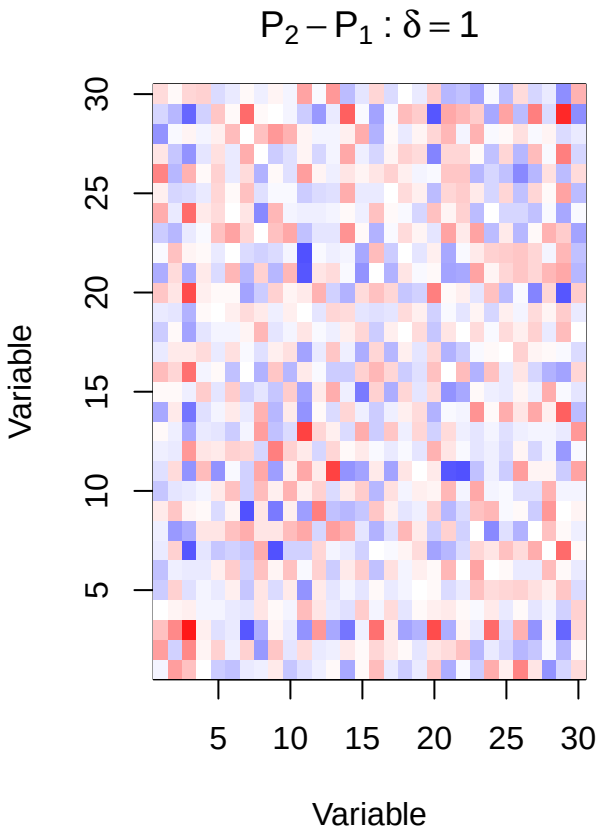

$P_2 - P_1 : \delta = 1$
Variable
5
10
15
20
25
30
Variable
5
10
15
20
25
30


$P_2 - P_1 : \delta = 0$
$P_2 - P_1 : \delta = 0.25$
$P_2 - P_1 : \delta = 0.5$
$P_2 - P_1 : \delta = 1$
Variable
Variable
5
10
15
20
25
30


$P_2 - P_1 : \delta = 0$
$P_2 - P_1 : \delta = 0.25$
Variable
Variable
5
10
15
20
25
30


$P_2 - P_1 : \delta = 0$
$P_2 - P_1 : \delta = 0.5$
Variable
Variable
5
10
15
20
25
30






```
# Load the package used to display tables.
library(knitr)
```

Now we set up functions to do our testing. Used the recomendations of the article. (UPDATE) added Bai-Ng ICp2 penalty, at the presentation it was edited out this Rmd has it included, none the less the inital analysis is left unchanged $\delta = 0, 0.5$ are still valid for the H_0

```
# Define the functions

# Estimate a loading-space basis using PCA
# Rows of X are observations; columns are variables
loading_basis <- function(X, r) {
  svd(X, nu = 0, nv = r)$v[, 1:r, drop = FALSE]
}


# Select the number of factors and estimate their values
estimate_factors <- function(Y, rmax = 8) {

  n <- nrow(Y)
  d <- ncol(Y)

  # Keep the candidate count below the matrix dimensions
  rmax <- min(rmax, n - 1, d - 1)
```

```r
  # Calculate the PCA decomposition once
  fit <- svd(Y, nu = rmax, nv = 0)

  # Compare models containing 0, 1, ..., rmax factors
  candidates <- 0:rmax
  IC <- numeric(length(candidates))

  # Bai-Ng ICp2 penalty for each additional factor
  penalty <- ((n + d) / (n * d)) * log(min(n, d))

  for (k in candidates) {

    # Average squared error remaining after retaining k factors
    residual_variance <- (
      sum(fit$d^2) - sum(fit$d[seq_len(k)]^2)
    ) / (n * d)

    # Prevent taking the logarithm of zero
    residual_variance <- max(
      residual_variance,
      .Machine$double.eps
    )

    IC[k + 1] <- log(residual_variance) + k * penalty
  }

  # Choose the factor count with the smallest criterion
  selected_r <- candidates[which.min(IC)]

  # Normalize factors so that t(F) %*% F / n = I
  F <- sqrt(n) *
    fit$u[, seq_len(selected_r), drop = FALSE]

  list(r = selected_r, F = F)
}


# Calculate the Wald statistic from estimated factors
# bandwidth controls how many serial-dependence lags receive weight
wald_test <- function(F, bandwidth = ceiling(nrow(F)^(1 / 3))) {

  n <- nrow(F)
  r <- ncol(F)

  stopifnot(n %% 2 == 0, r > 0)

  # Split the observations at the midpoint
  first <- 1:(n / 2)
  second <- (n / 2 + 1):n

  # Select the lower triangle, including the diagonal
  # These are the nonduplicated entries of F_t F_t'
  lower <- lower.tri(matrix(0, r, r), diag = TRUE)
```

```
  products <- do.call(
    rbind,
    lapply(1:n, function(t) {
      tcrossprod(F[t, ])[lower]
    })
  )

  # Compare factor second moments between the two halves
  V <- (
    colSums(products[first, , drop = FALSE]) -
      colSums(products[second, , drop = FALSE])
  ) / sqrt(n)

  # Subtract the identity entries, following the paper
  Z <- sweep(products, 2, diag(r)[lower], "-")

  # Start with the lag-zero covariance
  Omega <- crossprod(Z) / n

  # Add lagged covariances using Bartlett weights
  # For bandwidth = 6
  bandwidth <- max(1, min(bandwidth, n - 1))

  if (bandwidth > 1) {
    for (lag in 1:(bandwidth - 1)) {

      Gamma <- crossprod(
        Z[(lag + 1):n, , drop = FALSE],
        Z[1:(n - lag), , drop = FALSE]
      ) / n

      weight <- 1 - lag / bandwidth
      Omega <- Omega + weight * (Gamma + t(Gamma))
    }
  }

  # Stop rather than report an unreliable inverse
  if (rcond(Omega) < 1e-10) {
    stop("The estimated covariance is nearly singular.")
  }

  # Calculate the statistic and its chi-square p-value
  W <- as.numeric(t(V) %*% solve(Omega, V))
  df <- r * (r + 1) / 2
  p_value <- pchisq(W, df = df, lower.tail = FALSE)

  c(W = W, p_value = p_value, bandwidth = bandwidth)
}
```

Now we move on to the hypothesis testing.

```r
# State the hypothesis

cat(
  "H0: Lambda_k = Lambda Phi_k, for k = 1, 2.\n",
  "H1: The loading spaces differ.\n"
)
```

```
## H0: Lambda_k = Lambda Phi_k, for k = 1, 2.
##  H1: The loading spaces differ.
```

```r
# Compare the TRUE loading spaces from the simulation.
# This is a descriptive calculation, not the statistical test.
space_difference <- numeric(length(delta_values))

for (j in seq_along(delta_values)) {
  space_difference[j] <- sqrt(
    sum(projection_differences[[j]]^2)
  )
}

kable(
  data.frame(
    delta = delta_values,
    projection_difference = space_difference
  ),
  digits = 3,
  caption = "Differences between the true loading spaces"
)
```

Table 1: Differences between the true loading spaces

delta	projection_difference
0.00	0.000
0.25	0.618
0.50	1.138
1.00	1.786

We runn the proposed transformation of the factor model, where our models become a singular model but changes nature as T increases.

```r
# Transform the data and run the test

# T must be even so that both halves have equal length
stopifnot(T %% 2 == 0)

# Estimate Dataset 1's loading space
Q1 <- loading_basis(X1, r)

first <- 1:(T / 2)
second <- (T / 2 + 1):T
```

```r
results <- list()

for (j in seq_along(delta_values)) {

  # Select Dataset 2 for this delta
  X <- X2[[j]]

  # Estimate its loading space
  Q2 <- loading_basis(X, r)

  # Transform the first half using X1's projection
  Y <- X

  Y[first, ] <- (
    X[first, ] + X[first, ] %*% Q1 %*% t(Q1)
  ) / 2

  # Transform the second half using X2's projection
  Y[second, ] <- (
    X[second, ] + X[second, ] %*% Q2 %*% t(Q2)
  ) / 2

  # Select and estimate factors from the entire transformed series
  fit <- estimate_factors(Y, rmax = 8)

  W <- NA_real_
  p_value <- NA_real_
  bandwidth <- NA_real_

  # Run the test if at least one factor was selected
  if (fit$r > 0) {
    test <- wald_test(fit$F)

    W <- unname(test["W"])
    p_value <- unname(test["p_value"])
    bandwidth <- unname(test["bandwidth"])
  }

  results[[j]] <- data.frame(
    delta = delta_values[j],
    selected_factors = fit$r,
    bandwidth = bandwidth,
    df = fit$r * (fit$r + 1) / 2,
    W = W,
    p_value = p_value
  )
}
```

Our conclusion: We can infer that $\delta \leq 0.5$ the perturbation is close enough to assume we have similar stucture. It still is needed to see if what is the maximum δ where the H_0 no longer holds. Recall our critical value is $W = 12.592$

```r
# Display the conclusions

test_results <- do.call(rbind, results)

test_results$conclusion <- ifelse(
  is.na(test_results$p_value),
  "Test unavailable",
  ifelse(
    test_results$p_value < alpha,
    "Reject H0",
    "Do not reject H0"
  )
)

kable(
  test_results,
  digits = 3,
  caption = "Simulation results for each delta"
)
```

Table 2: Simulation results for each delta

delta	selected_factors	bandwidth	df	W	p_value	conclusion
0.00	3	6	6	3.515	0.742	Do not reject H0
0.25	3	6	6	3.429	0.753	Do not reject H0
0.50	6	6	21	32.926	0.047	Reject H0
1.00	6	6	21	33.018	0.046	Reject H0